CS344L - Artificial Intelligence Lab

LAB 06 TASK

Name: Hussnain Ahmad

Roll No: 242816

Class: BSCSeve-V-C

Course: Artificial Intelligence (CS344L)

Lab: Lab 6 - Markov Models

Task 1 - A Markov chain over four words (8 marks)

A typing predictor over the words the/lab/is/open. check_rows() validates the matrix, next_step() pushes a belief through it with two loops, forecast() builds the 0-5 step table, and find_stationary() runs power iteration until the largest change drops below 1e-10. A verification block pushes pi through the matrix once more.

Code (242816_Task1.py):

```
"""
CS344L - Artificial Intelligence Lab | Lab 6 - Markov Models
Task 1 - A Markov chain over four words  (roll 242816)

Model (in my own words):
  States           : the four words "the", "lab", "is", "open".
  Transition matrix: TRANSITION[cur][nxt] = P(next word is nxt | current word is cur).
  Initial belief   : START - we begin 100% sure the first word is "the".
"""

WORDS = ["the", "lab", "is", "open"]

TRANSITION = {
    "the":  {"the": 0.10, "lab": 0.60, "is": 0.20, "open": 0.10},
    "lab":  {"the": 0.10, "lab": 0.10, "is": 0.70, "open": 0.10},
    "is":   {"the": 0.30, "lab": 0.10, "is": 0.10, "open": 0.50},
    "open": {"the": 0.50, "lab": 0.20, "is": 0.20, "open": 0.10},
}

START = {"the": 1.00, "lab": 0.00, "is": 0.00, "open": 0.00}


def check_rows():
    # Print the total of every transition row and say whether the matrix
    # is valid. A valid chain needs every row to sum to 1.0.
    valid = True
    for word in WORDS:
        total = sum(TRANSITION[word][nxt] for nxt in WORDS)
        good = abs(total - 1.0) < 1e-9
        valid = valid and good
        print(f"   row {word}   sums to {total:.2f}   {'OK' if good else 'BAD'}")
    return valid


def next_step(distribution):
    # One step of the chain: for each possible next word, add up the
    # probability of reaching it from every current word. Two loops, no library.
    new = {}
    for nxt in WORDS:
        total = 0.0
        for cur in WORDS:
            total += distribution[cur] * TRANSITION[cur][nxt]
        new[nxt] = total
    return new


def forecast(start, steps):
    # Apply next_step repeatedly; return the belief at steps 0..steps.
```

```
    dist = dict(start)
    history = [dict(dist)]
    for _ in range(steps):
        dist = next_step(dist)
        history.append(dict(dist))
    return history


def find_stationary(tolerance=1e-10):
    # Power iteration: start from a uniform belief and keep stepping until
    # the largest change in any word drops below the tolerance.
    # Returns (stationary distribution, steps taken).
    current = {w: 1.0 / len(WORDS) for w in WORDS}
    step = 0
    while True:
        step += 1
        nxt = next_step(current)
        change = max(abs(nxt[w] - current[w]) for w in WORDS)
        current = nxt
        if change < tolerance:
            return current, step


def main():
    # 1. Validate the matrix.
    print("Checking the matrix:")
    check_rows()
    print()

    # 2. Forecast table, steps 0 to 5, starting from 'the'.
    print("Five steps starting from 'the':")
    print("    step" + "".join(f"{w:>9}" for w in WORDS))
    for i, dist in enumerate(forecast(START, 5)):
        print(f"{i:>8}" + "".join(f"{dist[w]:>9.4f}" for w in WORDS))
    print()

    # 3. Stationary distribution by power iteration.
    pi, n = find_stationary()
    print(f"Stationary distribution (after {n} steps):")
    print(f"   {'the':<6} {pi['the']:.4f}      {'lab':<6} {pi['lab']:.4f}")
    print(f"   {'is':<6} {pi['is']:.4f}      {'open':<6} {pi['
open']:.4f}")
    print()

    # 4. Verification: push pi through the matrix once more.
    print("Verify pi * P = pi:")
    back = next_step(pi)
    for w in WORDS:
        diff = abs(back[w] - pi[w])
        print(f"   {w:<6} pi={pi[w]:.4f}  (pi*P)={back[w]:.4f}  diff={diff:.4f}")


if __name__ == "__main__":
    main()
```

Output: run window of Task 1

```
Task 1 - Markov chain: forecast + stationary
$ python 242816_Task1.py
Checking the matrix:
   row the   sums to 1.00   OK
   row lab   sums to 1.00   OK
   row is   sums to 1.00   OK
   row open   sums to 1.00   OK

Five steps starting from 'the':
    step      the      lab       is     open
       0   1.0000   0.0000   0.0000   0.0000
       1   0.1000   0.6000   0.2000   0.1000
       2   0.1800   0.1600   0.4800   0.1800
       3   0.2680   0.2080   0.2320   0.2920
       4   0.2632   0.2632   0.2808   0.1928
       5   0.2333   0.2509   0.3035   0.2123

Stationary distribution (after 26 steps):
   the    0.2455      lab    0.2444
   is     0.2929      open   0.2172

Verify pi * P = pi:
   the    pi=0.2455  (pi*P)=0.2455  diff=0.0000
   lab    pi=0.2444  (pi*P)=0.2444  diff=0.0000
   is     pi=0.2929  (pi*P)=0.2929  diff=0.0000
   open   pi=0.2172  (pi*P)=0.2172  diff=0.0000
```

Q1. Look at step 1 in your table. Why is 'lab' at 0.60 and 'open' at only 0.10? Point at the exact row of the matrix that produces those numbers.

Step 0 is 100% on 'the', so step 1 is simply the 'the' row of the matrix copied out: P(lab | the) = 0.60 and P(open | the) = 0.10.

Q2. Run your forecast again starting from 'open' instead of 'the'. Do the first three steps change? Does the stationary distribution change? Explain the difference in two lines.

The first three steps are completely different, because the early forecast depends on where the chain starts. The stationary distribution is identical, because pi is a property of the transition matrix, not of the starting belief - the chain mixes and forgets its start.

Task 2 - Simulate the chain and verify pi (6 marks)

A real word sequence is generated with random.Random(42): pick() throws a dart on the 0-1 ruler with a running total, simulate() walks the chain using each word's transition row, and count_words() measures the fraction spent on each word. A fresh Random(42) is created for every table row.

Code (242816_Task2.py):

```
"""
CS344L - Artificial Intelligence Lab | Lab 6 - Markov Models
Task 2 - Simulate the chain and verify pi  (roll 242816)

Instead of calculating probabilities, we generate a real word sequence with
the same Markov chain from Task 1. If the maths is right, the fraction of
time the simulation spends on each word must approach Task 1's stationary
distribution as the run gets longer.
"""

import random

WORDS = ["the", "lab", "is", "open"]

TRANSITION = {
    "the":  {"the": 0.10, "lab": 0.60, "is": 0.20, "open": 0.10},
    "lab":  {"the": 0.10, "lab": 0.10, "is": 0.70, "open": 0.10},
    "is":   {"the": 0.30, "lab": 0.10, "is": 0.10, "open": 0.50},
    "open": {"the": 0.50, "lab": 0.20, "is": 0.20, "open": 0.10},
}

START = {"the": 1.00, "lab": 0.00, "is": 0.00, "open": 0.00}


def stationary(tolerance=1e-10):
    # Power iteration on the Task 1 chain: step a uniform belief until it
    # stops moving. Gives the pi we compare the simulation against.
    def next_step(distribution):
        new = {}
        for nxt in WORDS:
            total = 0.0
            for cur in WORDS:
                total += distribution[cur] * TRANSITION[cur][nxt]
            new[nxt] = total
        return new

    current = {w: 1.0 / len(WORDS) for w in WORDS}
    while True:
        nxt = next_step(current)
        if max(abs(nxt[w] - current[w]) for w in WORDS) < tolerance:
            return nxt
        current = nxt


def pick(distribution, rng):
    # Choose one word at random following the given probabilities: lay the
    # probabilities end to end on a 0..1 ruler, throw a dart with rng.random(),
    # and return whichever word's segment the dart lands in.
```

```
    dart = rng.random()
    running = 0.0
    for word in WORDS:
        running += distribution[word]
        if dart < running:
            return word
    return WORDS[-1]  # safety net for rounding


def simulate(steps, rng):
    # Generate a real sequence: the first word comes from START, every next
    # word is picked using the current word's transition row.
    sequence = [pick(START, rng)]
    for _ in range(steps - 1):
        sequence.append(pick(TRANSITION[sequence[-1]], rng))
    return sequence


def count_words(sequence):
    # The fraction of the sequence spent on each word.
    n = len(sequence)
    return {w: sequence.count(w) / n for w in WORDS}


def main():
    # 1. The first 12 words of one reproducible run.
    pi = stationary()
    print("First 12 words generated:")
    print("   " + " ".join(simulate(12, random.Random(42))))
    print()

    # 2. Counts for longer and longer runs, each with a fresh Random(42).
    print("Counts for longer and longer runs:")
    print("      steps" + "".join(f"{w:>9}" for w in WORDS) + "   max error")
    for steps in (100, 1000, 10000, 50000):
        seq = simulate(steps, random.Random(42))  # fresh rng for each row
        frac = count_words(seq)
        err = max(abs(frac[w] - pi[w]) for w in WORDS)
        print(f"{steps:>11}" + "".join(f"{frac[w]:>
9.4f}" for w in WORDS)
              + f"{err:>11.4f}")
    print(f"{'pi':>11}" + "".join(f"{pi[w]:>9.4f}" for w in WORDS)
          + "           -")


if __name__ == "__main__":
    main()
```

Output: run window of Task 2

```
Task 2 - Simulation vs stationary distribution
$ python 242816_Task2.py
First 12 words generated:
   the the lab is open lab is the lab the lab is

Counts for longer and longer runs:
      steps      the      lab       is     open   max error
        100   0.2700   0.2600   0.2600   0.2100     0.0329
       1000   0.2390   0.2400   0.2990   0.2220     0.0065
      10000   0.2464   0.2427   0.2956   0.2153     0.0027
      50000   0.2459   0.2450   0.2921   0.2169     0.0008
         pi   0.2455   0.2444   0.2929   0.2172          -
```

Q3. Copy your error column. What happens to the error as the number of steps grows, and why does it never reach exactly zero?

The max error falls steadily: 0.0329 at 100 steps, 0.0065 at 1,000, 0.0027 at 10,000 and 0.0008 at 50,000. It never reaches exactly zero because a simulation is a finite sample of a random process - pi is the limit the counts approach, not a value any finite run must hit.

Q4. Remove random.seed / random.Random(42) and run your file three times. Which numbers change and which stay roughly the same? What does that tell you about pi?

The 12 generated words change completely on every run, and the small-sample rows (100, 1,000 steps) move noticeably, while the 50,000-step row barely moves and always stays close to pi. That shows pi is a property of the model, not of any particular random run: individual sequences differ, long-run proportions do not.

Task 3 - A hidden Markov model (6 marks)

The machine is Working or Broken (hidden); we only see Good or Faulty items. forward() builds the alpha table with sum (day 0 = start x emission), filtering normalises each row into a daily belief, viterbi() uses max with back-pointers and walks backwards for the best path, and brute_force() checks all 2^4 = 16 hidden paths.

Code (242816_Task3.py):

```
"""
CS344L - Artificial Intelligence Lab | Lab 6 - Markov Models
Task 3 - A hidden Markov model  (roll 242816)

A factory machine is either Working or Broken, but we cannot see inside it -
we only observe whether each item it produces is Good or Faulty. The hidden
states follow a Markov chain (TRANSITION) and each state emits observations
with its own probabilities (EMISSION). We run the forward algorithm (sum over
all paths) and Viterbi (best single path) on four days of evidence, then
check both against a brute-force search over all 2^4 = 16 hidden paths.
"""

from itertools import product

STATES = ["Working", "Broken"]
OBSERVATIONS = ["Good", "Faulty"]

START = {"Working": 0.80, "Broken": 0.20}

TRANSITION = {
    "Working": {"Working": 0.85, "Broken": 0.15},
    "Broken":  {"Working": 0.30, "Broken": 0.70},
}

EMISSION = {
    "Working": {"Good": 0.90, "Faulty": 0.10},
    "Broken":  {"Good": 0.25, "Faulty": 0.75},
}

EVIDENCE = ["Good", "Faulty", "Faulty", "Good"]


def check_rows():
    # Every transition row and every emission row must sum to 1.0.
    valid = True
    for s in STATES:
        t = sum(TRANSITION[s][nxt] for nxt in STATES)
        e = sum(EMISSION[s][o] for o in OBSERVATIONS)
        good = abs(t - 1.0) < 1e-9 and abs(e - 1.0) < 1e-9
        valid = valid and good
        print(f"   {s:<8} transition sums to {t:.2f}, "
              f"emission sums to {e:.2f}   {'OK' if good else 'BAD'}")
    return valid


def forward(evidence):
```

```
    # alpha[day][state] = P(hidden states so far, evidence so far).
    # Day 0: start times emission. Every later day: add up the probability
    # of arriving from each previous state, then multiply by the emission
    # of what was actually seen. Returns (alpha table, P(evidence)).
    alpha = {s: START[s] * EMISSION[s][evidence[0]] for s in STATES}
    table = [dict(alpha)]
    for day in range(1, len(evidence)):
        new = {}
        for current in STATES:
            arriving = sum(alpha[prev] * TRANSITION[prev][current]
                           for prev in STATES)          # sum over paths
            new[current] = arriving * EMISSION[current][evidence[day]]
        alpha = new
        table.append(dict(alpha))
    return table, sum(alpha.values())


def filter_beliefs(table):
    # Turn each alpha row into a belief: divide by the row's own total.
    beliefs = []
    for row in table:
        total = sum(row.values())
        beliefs.append({s: row[s] / total for s in STATES})
    return beliefs


def viterbi(evidence):
    # Like forward, but max instead of sum: keep only the best way of
    # reaching each state, remember where it came from (back-pointers),
    # then walk backwards from the best final state. Returns (path, prob).
    score = {s: START[s] * EMISSION[s][evidence[0]] for s in STATES}
    came_from = [{s: None for s in STATES}]
    for day in range(1, len(evidence)):
        new_score = {}
        new_from = {}
        for current in STATES:
            best_prev = max
(STATES,
                           key=lambda p: score[p] * TRANSITION[p][current])
            new_score[current] = (score[best_prev]
                                  * TRANSITION[best_prev][current]
                                  * EMISSION[current][evidence[day]])
            new_from[current] = best_prev
        score = new_score
        came_from.append(new_from)
    last = max(STATES, key=lambda s: score[s])
    prob = score[last]
    path = [last]
    for day in range(len(evidence) - 1, 0, -1):
        last = came_from[day][last]
        path.append(last)
    path.reverse()
    return path, prob


def brute_force(evidence):
    # Try every one of the 2^4 = 16 hidden paths directly.
    # Returns (total probability, best path, best path probability).
    total = 0.0
    best_path = None
```

```
    best_prob = -1.0
    for path in product(STATES, repeat=len(evidence)):
        p = START[path[0]] * EMISSION[path[0]][evidence[0]]
        for day in range(1, len(evidence)):
            p *= (TRANSITION[path[day - 1]][path[day]]
                  * EMISSION[path[day]][evidence[day]])
        total += p
        if p > best_prob:
            best_prob = p
            best_path = list(path)
    return total, best_path, best_prob


def main():
    # 1. Validate both matrices.
    print("Checking the model:")
    check_rows()
    print()

    # 2. Forward algorithm.
    table, p_evidence = forward(EVIDENCE)
    print("Forward algorithm")
    print("    day      saw   alpha Working    alpha Broken")
    for day, row in enumerate(table):
        print(f"{day:>7}  {EVIDENCE[day]:>7}"
              f"   {row['Working']:.4f}         {row['Broken']:.4f}")
    print(f"   P(evidence) = {p_evidence:.6f}")
    print()

    # 3. Filtering: belief about each day, with the most likely state.
    print("Filtering (belief about each day)")
    print("    day      saw    Working     Broken   most likely")
    for day, belief in enumerate(filter_beliefs(table)):
        w, b = belief["Working"], belief["Broken"]
        likely = "Working" if w >= b else "Broken"
        print(f"{day:>7}  {EVIDENCE[day]:>7}"
              f"     {w:.4f}     {b:.4f}   {likely}")
    print()

    # 4. Viterbi: the single best hidden path.
    path, prob = viterbi(EVIDENCE)
    print("Viterbi")
    print(f"   best path        : {path}")
    print(f"   its probability  : {prob:.6f}")
    print()

    # 5. Brute-force check over all 16 paths.
    total, best_path, best_prob = brute_force(EVIDENCE)
    same = (abs(total - p_evidence) < 1e-9 and best_path == path
            and abs(best_prob - prob) < 1e-9)
    print("Check against all 16 paths")
    print(f"   same answer : {same}")


if __name__ == "__main__":
    main()
```

Output: run window of Task 3

```
Task 3 - HMM: forward + Viterbi
$ python 242816_Task3.py
Checking the model:
   Working  transition sums to 1.00, emission sums to 1.00   OK
   Broken   transition sums to 1.00, emission sums to 1.00   OK

Forward algorithm
    day      saw   alpha Working    alpha Broken
      0     Good   0.7200         0.0500
      1   Faulty   0.0627         0.1073
      2   Faulty   0.0085         0.0634
      3     Good   0.0236         0.0114
   P(evidence) = 0.035054

Filtering (belief about each day)
    day      saw    Working     Broken   most likely
      0     Good     0.9351     0.0649   Working
      1   Faulty     0.3689     0.6311   Broken
      2   Faulty     0.1189     0.8811   Broken
      3     Good     0.6745     0.3255   Working

Viterbi
   best path        : ['Working', 'Broken', 'Broken', 'Working']
   its probability  : 0.011482

Check against all 16 paths
   same answer : True
```

Q5. On day 3 the machine produced a Good item after two Faulty ones. What does your filtering row say for day 3, and why is the belief not back to where it was on day 0?

Day 3 filtering gives Working 0.6745 and Broken 0.3255. The belief swings back towards Working because a Good item is far more likely from a working machine (0.90 against 0.25). It does not return to day 0's 0.9351 because the two Faulty days

left real evidence behind, and a broken machine stays broken 70% of the time - some of that doubt carries forward.

Q6. Compare P(evidence) from the forward algorithm with the probability of the single best Viterbi path. What percentage of the total does the best path carry, and what does that percentage tell you about how confident we should be?

P(evidence) = 0.035054 over all 16 paths; the best path (Working, Broken, Broken, Working) carries 0.011482, which is 32.8% of the total. So even the most likely explanation accounts for only about a third of the probability - it is the best single story, not a certain one; the other 67% is spread over the 15 remaining paths.

Notes

All three files run on their own with plain Python 3 (standard library only, no packages installed). Full project on GitHub: https://github.com/hussnainahmedd/hussnain-ai-lab-6